

GUIA MODULO III

Interfaz Grafica con C#

Windows Forms, eventos, controles, validaciones y CRUD funcional

Objetivo general

Desarrollar aplicaciones de escritorio con interfaz grafica en C# utilizando Windows Forms, comprendiendo el modelo de programacion por eventos, el uso de controles visuales, la separacion de responsabilidades y la construccion de un CRUD funcional con datos en memoria y persistencia en archivo.

Competencias a desarrollar

- Comprender la diferencia entre una aplicacion de consola y una aplicacion grafica.
- Crear formularios, controles, botones, cajas de texto, tablas visuales y mensajes.
- Programar eventos como Click, TextChanged, Load y CellClick.
- Validar datos antes de guardarlos.
- Organizar el codigo en Models, Services y Forms.
- Construir un sistema funcional de registro de productos usando C# y WinForms.

Indice de contenido

1. Introduccion a las interfaces graficas
2. Aplicaciones de consola vs aplicaciones graficas
3. Modelo de programacion por eventos
4. Windows Forms en C#
5. Controles principales y propiedades
6. Formularios, eventos y ciclo de vida
7. Validacion de datos en interfaces
8. Diseno de una interfaz ordenada
9. Organizacion de proyecto: Models, Services y Forms
10. Guia funcional: sistema CRUD de productos
11. Explicacion completa del codigo
12. Laboratorios guiados
13. Ejercicios propuestos
14. Rubrica de evaluacion
15. Recomendaciones finales

1. Introduccion a las interfaces graficas

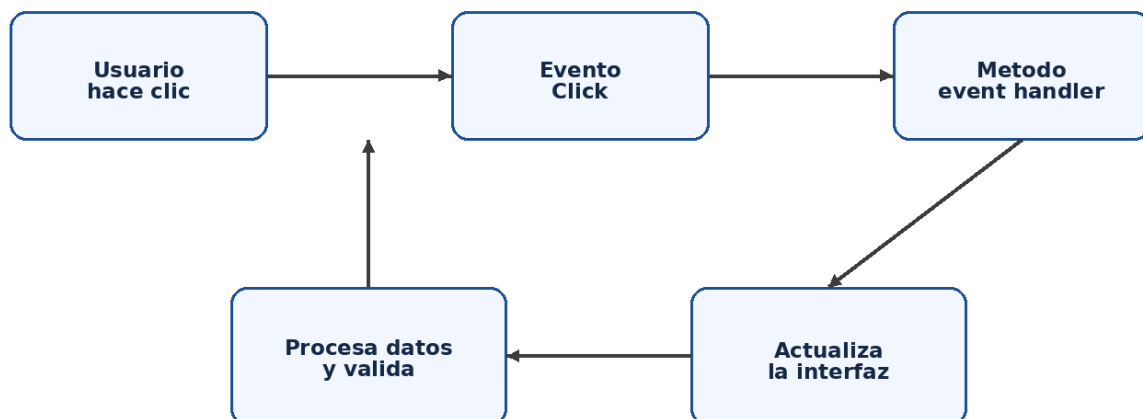
Una interfaz grafica de usuario, conocida como GUI por sus siglas en ingles Graphical User Interface, permite que una persona interactue con un programa mediante ventanas, botones, campos de texto, menus, listas, tablas y mensajes visuales. A diferencia de la consola, donde el usuario escribe instrucciones linea por linea, en una interfaz grafica el usuario observa opciones, selecciona elementos y ejecuta acciones mediante clics o entradas de texto.

En C#, una de las formas mas directas para iniciar con interfaces graficas de escritorio es Windows Forms. Esta tecnologia permite crear aplicaciones de ventanas para Windows y es muy util para estudiantes que ya dominan variables, condicionales, ciclos, clases, listas, archivos y bases de datos, porque conecta esos temas con una forma visual de presentar y manipular informacion.

La interfaz grafica no reemplaza la logica de programacion. Lo que cambia es la forma en que el usuario se comunica con el sistema. La logica sigue existiendo: se validan datos, se crean objetos, se guardan registros, se consultan listas y se muestran resultados. La diferencia es que ahora esas acciones se activan con eventos visuales.

Concepto	Descripcion	Ejemplo en C#
Ventana o formulario	Pantalla principal donde se colocan controles.	MainForm
Control	Elemento visual que permite mostrar o capturar informacion.	TextBox, Button, DataGridView
Evento	Accion que ocurre por interaccion del usuario o del sistema.	Click, Load, TextChanged
Metodo manejador	Metodo que se ejecuta cuando ocurre un evento.	btnGuardar_Click
Logica de negocio	Reglas que determinan como se procesan los datos.	No permitir precios negativos

Funcionamiento de una aplicacion grafica por eventos



Idea clave: en interfaz grafica el programa espera acciones del usuario y responde con eventos.

Diagrama 1. La interfaz grafica trabaja mediante eventos que activan metodos.

2. Aplicaciones de consola vs aplicaciones graficas

Una aplicacion de consola es ideal para aprender logica, estructuras de control y programacion orientada a objetos. Sin embargo, cuando un sistema se acerca mas al uso real por parte de usuarios finales, se vuelve necesario presentar la informacion de manera mas clara. Para eso se utilizan interfaces graficas.

Aspecto	Consola	Interfaz grafica
Entrada de datos	Console.ReadLine()	TextBox, ComboBox, DateTimePicker
Salida de datos	Console.WriteLine()	Label, MessageBox, DataGridView
Flujo	Secuencial, paso a paso	Por eventos, segun acciones del usuario
Facilidad de uso	Requiere escribir comandos o datos	Permite seleccionar, hacer clic y visualizar
Organizacion visual	Limitada	Alta: ventanas, paneles, tablas, botones
Uso comun	Practicas, scripts, pruebas iniciales	Sistemas administrativos, inventarios, ventas

El cambio mas importante es mental: en consola el programador suele controlar el orden exacto de ejecucion; en interfaz grafica el programa permanece abierto esperando acciones. Por eso el codigo se divide en eventos. Cada boton o control ejecuta una parte especifica del programa cuando el usuario interactua con el.

3. Modelo de programacion por eventos

La programacion por eventos consiste en crear metodos que se ejecutan como respuesta a una accion. Por ejemplo, cuando el usuario presiona el boton Guardar, se ejecuta el evento Click de ese boton. Cuando cambia el contenido de una caja de texto, puede ejecutarse TextChanged. Cuando el formulario aparece por primera vez, puede ejecutarse Load.

Evento	Cuando ocurre	Uso comun
Load	Cuando se abre el formulario	Cargar datos iniciales
Click	Cuando se hace clic sobre un boton	Guardar, eliminar, limpiar, buscar
TextChanged	Cuando cambia el texto de un TextBox	Validar o filtrar en tiempo real
SelectedIndexChanged	Cuando cambia la seleccion de un ComboBox	Actualizar datos relacionados
CellClick	Cuando se selecciona una celda de un DataGridView	Cargar datos en el formulario

```
private void btnGuardar_Click(object sender, EventArgs e)
{
    MessageBox.Show("Se hizo clic en el boton Guardar.");
}
```

En el ejemplo anterior, el metodo se ejecuta cuando el usuario hace clic en el boton. El parametro sender representa el control que provoco el evento y EventArgs contiene informacion adicional del evento. En ejercicios iniciales no siempre se usan, pero forman parte de la firma normal de los eventos en Windows Forms.

4. Windows Forms en C#

Windows Forms es una tecnología de .NET para crear aplicaciones de escritorio basadas en ventanas. Permite crear formularios visuales con controles, asociar eventos y ejecutar código C#. Aunque existen otras tecnologías como WPF, MAUI y Avalonia, Windows Forms sigue siendo una excelente opción educativa porque es directa, clara y permite comprender rápidamente como se conecta una interfaz con la lógica del programa.

Tecnología	Uso principal	Ventaja	Cuándo usarla
Windows Forms	Aplicaciones de escritorio Windows	Fácil de aprender y rápida para CRUD	Primeras aplicaciones gráficas
WPF	Escritorio Windows avanzado	Mejor separación visual con XAML	Interfaces más modernas
MAUI	Aplicaciones multiplataforma	Puede crear apps para varias plataformas	Proyectos modernos multiplataforma
Avalonia	Escritorio multiplataforma	Similar a WPF, abierto y flexible	Windows, Linux y macOS

En esta guía se trabajará con Windows Forms porque permite enfocarse en lo esencial: controles, eventos, formularios, validaciones y conexión con clases propias. La idea es que el estudiante aprenda a crear una aplicación gráfica funcional sin depender completamente del diseñador visual.

5. Preparación del entorno

Para trabajar con Windows Forms se recomienda usar Visual Studio Community en Windows, porque incluye diseñador visual. También puede usarse Visual Studio Code, pero para WinForms suele ser más cómodo Visual Studio. Si se trabaja con .NET moderno, se puede crear el proyecto desde terminal.

```
dotnet new winforms -n SistemaProductosGUI
cd SistemaProductosGUI
dotnet run
```

El comando `dotnet new winforms` crea un proyecto de Windows Forms. El archivo `Program.cs` inicia la aplicación y abre el formulario principal. El formulario principal normalmente hereda de `Form`, lo cual significa que obtiene el comportamiento de una ventana.

6. Controles principales de Windows Forms

Control	Función	Ejemplo de propiedad importante
Label	Muestra texto informativo	Text
TextBox	Permite escribir texto	Text
Button	Ejecuta acciones al hacer clic	Click
ComboBox	Permite seleccionar una opción	SelectedItem
CheckBox	Permite marcar verdadero o falso	Checked
RadioButton	Permite elegir una opción dentro de un grupo	Checked
DataGridView	Muestra datos en forma de tabla	DataSource
Panel	Agrupar controles visualmente	Controls
MessageBox	Muestra mensajes al usuario	Show()

Cada control tiene propiedades. Las propiedades definen como se ve o se comporta el control. Por ejemplo, Text define el texto visible, Name identifica el control dentro del código, Size define el tamaño y Location define la posición.

```
TextBox txtNombre = new TextBox();
txtNombre.Name = "txtNombre";
txtNombre.Location = new Point(120, 40);
txtNombre.Width = 200;
this.Controls.Add(txtNombre);
```

7. Formularios y ciclo de vida

Un formulario es una clase que hereda de Form. Al abrirse, se ejecutan procesos internos de inicialización. En proyectos creados con diseñador, existe un método InitializeComponent que genera los controles visuales. En esta guía se hará de forma programática para que el estudiante pueda ver que la interfaz también es código C#.

```
public partial class MainForm : Form
{
    public MainForm()
    {
        InitializeComponent();
    }
}
```

Cuando se usa diseñador, no se recomienda modificar manualmente todo el contenido generado automáticamente porque Visual Studio puede sobrescribirlo. Para fines educativos y control total, también se puede crear una clase Form y agregar los controles en código.

8. Diseño de una interfaz ordenada

Una interfaz gráfica debe ser funcional, clara y fácil de usar. No basta con colocar botones en la ventana. El programador debe pensar en la experiencia del usuario: qué datos se piden, en qué orden se piden, qué botones son necesarios y qué mensajes se mostrarán cuando algo salga bien o mal.

- Los campos deben tener etiquetas claras.
- Los botones deben tener nombres de acción: Agregar, Actualizar, Eliminar, Limpiar.
- La tabla debe mostrar los datos principales.
- Los mensajes deben explicar el error y cómo corregirlo.
- No se debe saturar la pantalla con controles innecesarios.
- Debe mantenerse el mismo estilo visual en todo el formulario.

Diseno visual sugerido: formulario de productos

Sistema de productos

Codigo

Nombre

Precio

Stock

Codigo	Nombre	Precio	Stock
P001	Teclado	15.00	8
P002	Mouse	8.50	15

Agregar

Actualizar

Eliminar

Limpiar

Diagrama 2. Propuesta de interfaz para un formulario CRUD de productos.

9. Organizacion del proyecto

Para evitar que todo el codigo quede mezclado en el formulario, se recomienda separar el proyecto en carpetas. Esta organizacion ayuda a que el programa sea mas facil de leer, mantener y ampliar.

Arquitectura recomendada para proyectos WinForms

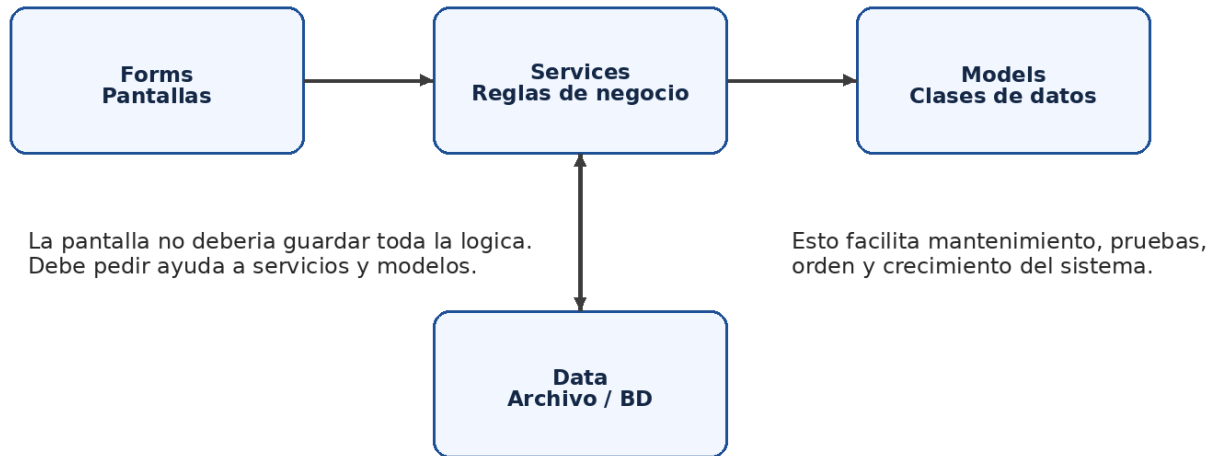


Diagrama 3. Separacion de responsabilidades entre Forms, Services, Models y Data.

```

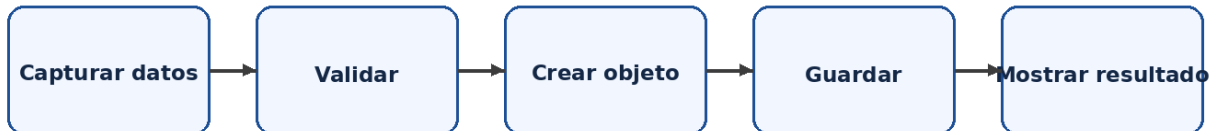
SistemaProductosGUI/
|
|-- Program.cs
|-- Forms/
|   |-- MainForm.cs
|-- Models/
|   |-- Product.cs
|-- Services/
|   |-- ProductService.cs
|-- Data/
|   |-- products.txt
  
```

Carpeta	Responsabilidad
Forms	Contiene las pantallas y eventos visuales.
Models	Contiene las clases que representan datos del sistema.
Services	Contiene reglas de negocio y operaciones principales.
Data	Contiene archivos o configuraciones de almacenamiento.

10. Proyecto funcional: Sistema de productos

El proyecto que se desarrollara en esta guia permite registrar productos mediante una interfaz grafica. El usuario podra agregar productos, verlos en una tabla, seleccionar un producto, actualizarlo, eliminarlo y guardar la informacion en un archivo de texto. Este proyecto aplica POO, listas, validaciones, archivos, eventos y controles graficos.

Flujo CRUD desde la interfaz grafica



CRUD significa Create, Read, Update y Delete. En WinForms normalmente se activa con botones, menus o dol

Diagrama 4. Flujo CRUD aplicado al sistema de productos.

10.1 Modelo Product.cs

El modelo representa la informacion de un producto. No contiene botones ni controles visuales. Su responsabilidad es definir los datos que tendra cada producto.

```

namespace SistemaProductosGUI.Models;

public class Product
{
    public string Code { get; set; } = string.Empty;
    public string Name { get; set; } = string.Empty;
    public decimal Price { get; set; }
    public int Stock { get; set; }

    public decimal TotalValue => Price * Stock;
}
  
```

La propiedad TotalValue es calculada. No se escribe directamente desde el formulario, sino que se obtiene multiplicando Price por Stock. Este tipo de propiedad es util para mostrar informacion adicional sin obligar al usuario a ingresarla manualmente.

10.2 Servicio ProductService.cs

El servicio administra la lista de productos. Aqui se colocan las reglas principales: agregar, actualizar, eliminar, buscar, cargar y guardar. De esta manera el formulario no queda saturado con logica de negocio.

```

using SistemaProductosGUI.Models;
using System.Globalization;

namespace SistemaProductosGUI.Services;

public class ProductService
{
    private readonly List<Product> products = new();
    private readonly string filePath = Path.Combine("Data", "products.txt");

    public List<Product> GetAll()
    {
        return products.ToList();
    }

    public bool Exists(string code)
    {
        return products.Any(p => p.Code.Equals(code, StringComparison.OrdinalIgnoreCase));
    }

    public void Add(Product product)
    {
        if (Exists(product.Code))
            throw new Exception("Ya existe un producto con ese codigo.");

        products.Add(product);
        SaveToFile();
    }

    public void Update(Product product)
    {
        Product? current = products.FirstOrDefault(p =>
            p.Code.Equals(product.Code, StringComparison.OrdinalIgnoreCase));

        if (current == null)
            throw new Exception("No se encontro el producto a actualizar.");

        current.Name = product.Name;
        current.Price = product.Price;
        current.Stock = product.Stock;
        SaveToFile();
    }

    public void Delete(string code)
    {
        Product? product = products.FirstOrDefault(p =>
            p.Code.Equals(code, StringComparison.OrdinalIgnoreCase));

        if (product == null)
            throw new Exception("No se encontro el producto a eliminar.");

        products.Remove(product);
        SaveToFile();
    }

    public void LoadFromFile()

```

```

{
    products.Clear();

    if (!File.Exists(filePath))
        return;

    foreach (string line in File.ReadAllLines(filePath))
    {
        string[] data = line.Split('|');

        if (data.Length != 4)
            continue;

        products.Add(new Product
        {
            Code = data[0],
            Name = data[1],
            Price = decimal.Parse(data[2], CultureInfo.InvariantCulture),
            Stock = int.Parse(data[3])
        });
    }
}

private void SaveToFile()
{
    Directory.CreateDirectory("Data");

    List<string> lines = products.Select(p =>
        $"{p.Code}|{p.Name}|{p.Price.ToString(CultureInfo.InvariantCulture)}|{p.Stock}").ToList();

    File.WriteAllLines(filePath, lines);
}
}

```

10.3 Program.cs

Program.cs es el punto de entrada. En una aplicacion grafica no se usa Console.WriteLine para iniciar la interaccion. En su lugar, se habilitan estilos visuales y se abre el formulario principal.

```

using SistemaProductosGUI.Forms;

namespace SistemaProductosGUI;

internal static class Program
{
    [STAThread]
    static void Main()
    {
        ApplicationConfiguration.Initialize();
        Application.Run(new MainForm());
    }
}

```

El atributo [STAThread] es comun en aplicaciones Windows Forms porque varios componentes visuales de Windows requieren ejecutarse bajo ese modelo de subprocessos. Application.Run mantiene abierta la aplicacion hasta que se cierre el formulario principal.

10.4 Formulario MainForm.cs

El formulario contiene los controles visuales y los eventos. En este ejemplo se crea todo mediante código para que sea posible copiar y pegar sin depender del diseñador visual.

```
using SistemaProductosGUI.Models;
using SistemaProductosGUI.Services;

namespace SistemaProductosGUI.Forms;

public class MainForm : Form
{
    private readonly ProductService service = new();

    private TextBox txtCode = new();
    private TextBox txtName = new();
    private TextBox txtPrice = new();
    private TextBox txtStock = new();
    private DataGridView dgvProducts = new();

    private Button btnAdd = new();
    private Button btnUpdate = new();
    private Button btnDelete = new();
    private Button btnClear = new();

    public MainForm()
    {
        ConfigureForm();
        CreateControls();
        ConfigureEvents();

        service.LoadFromFile();
        RefreshTable();
    }

    private void ConfigureForm()
    {
        Text = "Sistema de Productos";
        Width = 950;
        Height = 600;
        StartPosition = FormStartPosition.CenterScreen;
        FormBorderStyle = FormBorderStyle.FixedSingle;
        MaximizeBox = false;
    }

    private void CreateControls()
    {
        Label lblTitle = new Label
        {
            Text = "Registro de productos",
            Left = 25,
            Top = 20,
            Width = 400,
            Font = new Font("Segoe UI", 16, FontStyle.Bold)
        };
    }
};
```

```

Label lblCode = CreateLabel("Codigo:", 30, 80);
txtCode = CreateTextBox(130, 80);

Label lblName = CreateLabel("Nombre:", 30, 125);
txtName = CreateTextBox(130, 125);

Label lblPrice = CreateLabel("Precio:", 30, 170);
txtPrice = CreateTextBox(130, 170);

Label lblStock = CreateLabel("Stock:", 30, 215);
txtStock = CreateTextBox(130, 215);

btnAdd = CreateButton("Agregar", 30, 280);
btnUpdate = CreateButton("Actualizar", 150, 280);
btnDelete = CreateButton("Eliminar", 290, 280);
btnClear = CreateButton("Limpiar", 410, 280);

dgvProducts.Left = 30;
dgvProducts.Top = 350;
dgvProducts.Width = 870;
dgvProducts.Height = 170;
dgvProducts.ReadOnly = true;
dgvProducts.SelectionMode = DataGridViewSelectionMode.FullRowSelect;
dgvProducts.AutoSizeColumnsMode = DataGridViewAutoSizeColumnsMode.Fill;
dgvProducts.AllowUserToAddRows = false;

Controls.AddRange(new Control[]
{
    lblTitle, lblCode, txtCode, lblName, txtName,
    lblPrice, txtPrice, lblStock, txtStock,
    btnAdd, btnUpdate, btnDelete, btnClear, dgvProducts
});
}

private Label CreateLabel(string text, int left, int top)
{
    return new Label
    {
        Text = text,
        Left = left,
        Top = top + 5,
        Width = 90,
        Font = new Font("Segoe UI", 10, FontStyle.Bold)
    };
}

private TextBox CreateTextBox(int left, int top)
{
    return new TextBox
    {
        Left = left,
        Top = top,
        Width = 250,
        Font = new Font("Segoe UI", 10)
    };
}

```

```

private Button CreateButton(string text, int left, int top)
{
    return new Button
    {
        Text = text,
        Left = left,
        Top = top,
        Width = 110,
        Height = 40,
        Font = new Font("Segoe UI", 10, FontStyle.Bold)
    };
}

private void ConfigureEvents()
{
    btnAdd.Click += BtnAdd_Click;
    btnUpdate.Click += BtnUpdate_Click;
    btnDelete.Click += BtnDelete_Click;
    btnClear.Click += BtnClear_Click;
    dgvProducts.CellClick += DgvProducts_CellClick;
}

private void BtnAdd_Click(object? sender, EventArgs e)
{
    try
    {
        Product product = ReadProductFromForm();
        service.Add(product);
        RefreshTable();
        ClearFields();
        MessageBox.Show("Producto agregado correctamente.");
    }
    catch (Exception ex)
    {
        MessageBox.Show(ex.Message, "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}

private void BtnUpdate_Click(object? sender, EventArgs e)
{
    try
    {
        Product product = ReadProductFromForm();
        service.Update(product);
        RefreshTable();
        ClearFields();
        MessageBox.Show("Producto actualizado correctamente.");
    }
    catch (Exception ex)
    {
        MessageBox.Show(ex.Message, "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}

private void BtnDelete_Click(object? sender, EventArgs e)

```

```

{
    try
    {
        if (string.IsNullOrEmpty(txtCode.Text))
            throw new Exception("Seleccione o escriba el codigo del producto.");

        service.Delete(txtCode.Text.Trim());
        RefreshTable();
        ClearFields();
        MessageBox.Show("Producto eliminado correctamente.");
    }
    catch (Exception ex)
    {
        MessageBox.Show(ex.Message, "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}

private void BtnClear_Click(object? sender, EventArgs e)
{
    ClearFields();
}

private void DgvProducts_CellClick(object? sender, DataGridViewCellEventArgs e)
{
    if (e.RowIndex < 0)
        return;

    DataGridViewRow row = dgvProducts.Rows[e.RowIndex];

    txtCode.Text = row.Cells["Code"].Value?.ToString();
    txtName.Text = row.Cells["Name"].Value?.ToString();
    txtPrice.Text = row.Cells["Price"].Value?.ToString();
    txtStock.Text = row.Cells["Stock"].Value?.ToString();
}

private Product ReadProductFromForm()
{
    if (string.IsNullOrEmpty(txtCode.Text))
        throw new Exception("El codigo es obligatorio.");

    if (string.IsNullOrEmpty(txtName.Text))
        throw new Exception("El nombre es obligatorio.");

    if (!decimal.TryParse(txtPrice.Text, out decimal price) || price <= 0)
        throw new Exception("El precio debe ser un numero mayor que cero.");

    if (!int.TryParse(txtStock.Text, out int stock) || stock < 0)
        throw new Exception("El stock debe ser un numero entero mayor o igual a cero.");

    return new Product
    {
        Code = txtCode.Text.Trim(),
        Name = txtName.Text.Trim(),
        Price = price,
        Stock = stock
    };
}

```



```

    }

    private void RefreshTable()
    {
        dgvProducts.DataSource = null;
        dgvProducts.DataSource = service.GetAll();
    }

    private void ClearFields()
    {
        txtCode.Clear();
        txtName.Clear();
        txtPrice.Clear();
        txtStock.Clear();
        txtCode.Focus();
    }
}

```

11. Explicacion completa del codigo

Linea o elemento	Explicacion
ProductService service = new();	Crea un objeto de servicio que administra la lista de productos y el archivo donde se guardan.
TextBox txtCode = new();	Declara una caja de texto para ingresar el codigo del producto.
ConfigureForm();	Configura las propiedades generales de la ventana.
CreateControls();	Crea etiquetas, cajas de texto, botones y la tabla.
ConfigureEvents();	Conecta cada boton con su metodo de evento.
service.LoadFromFile();	Carga los productos guardados anteriormente en el archivo.
RefreshTable();	Actualiza la tabla visual con la informacion actual.
btnAdd.Click += BtnAdd_Click;	Indica que al hacer clic en Agregar se ejecutara BtnAdd_Click.
ReadProductFromForm();	Lee los datos escritos en pantalla y valida que sean correctos.
decimal.TryParse	Intenta convertir el texto del precio a decimal sin que el programa se cierre.
DataGridView	Control visual que muestra informacion en filas y columnas.
MessageBox.Show	Muestra mensajes al usuario cuando ocurre una accion o error.

La parte mas importante para comprender el codigo es identificar que el formulario no debe hacer todo. El formulario captura datos y responde a eventos. El servicio administra la informacion. El modelo representa el producto. Esa separacion permite que el sistema crezca sin volverse desordenado.

12. Validaciones en interfaces graficas

Validar datos es obligatorio en una aplicacion grafica. Aunque el usuario vea campos y botones, puede escribir informacion incorrecta: precios negativos, campos vacios, letras donde se esperan numeros o codigos repetidos. Por eso antes de guardar se debe revisar cada dato.

Validacion	Ejemplo	Mensaje sugerido
Campo obligatorio	Codigo vacio	El codigo es obligatorio.
Numero decimal	Precio = abc	El precio debe ser numerico.
Valor positivo	Precio = -5	El precio debe ser mayor que cero.
Numero entero	Stock = 2.5	El stock debe ser entero.
Codigo unico	P001 repetido	Ya existe un producto con ese codigo.

```
if (string.IsNullOrEmpty(txtName.Text))
{
    MessageBox.Show("El nombre es obligatorio.");
    txtName.Focus();
    return;
}
```

Focus sirve para regresar el cursor al campo que debe corregirse. Esto mejora la experiencia del usuario porque no solo se indica el error, sino que se ubica rapidamente donde debe corregirlo.

13. Buenas practicas al crear interfaces

- Usar nombres claros para controles: txtName, btnAdd, dgvProducts.
- Evitar nombres genericos como textBox1 o button1 en proyectos finales.
- No mezclar toda la logica dentro del evento Click.
- Mostrar mensajes claros y no tecnicos para el usuario.
- Limpiar campos despues de guardar correctamente.
- Bloquear acciones peligrosas con confirmacion, por ejemplo eliminar.
- Organizar visualmente los controles por secciones.
- Probar entradas incorrectas antes de entregar el sistema.

```
DialogResult result = MessageBox.Show(
    "¿Esta seguro de eliminar este producto?",
    "Confirmar eliminacion",
    MessageBoxButtons.YesNo,
    MessageBoxIcon.Question);

if (result == DialogResult.No)
    return;
```

14. Laboratorio guiado 1: crear el proyecto

1. Abrir Visual Studio o una terminal en la carpeta de trabajo.
2. Crear el proyecto con `dotnet new winforms -n SistemaProductosGUI`.
3. Crear las carpetas Models, Services, Forms y Data.
4. Agregar la clase Product dentro de Models.
5. Agregar ProductService dentro de Services.
6. Agregar MainForm dentro de Forms.
7. Modificar Program.cs para abrir MainForm.
8. Ejecutar el proyecto con `dotnet run`.
9. Probar agregar, actualizar, eliminar y limpiar productos.

Resultado esperado: el estudiante debe obtener una ventana con campos para codigo, nombre, precio y stock, botones funcionales y una tabla que muestre los productos registrados.

15. Laboratorio guiado 2: agregar busqueda

El siguiente paso consiste en agregar una caja de busqueda para filtrar productos por nombre o codigo. Esto permite practicar el evento `TextChanged`.

```
private TextBox txtSearch = new();

private void ConfigureEvents()
{
    btnAdd.Click += BtnAdd_Click;
    btnUpdate.Click += BtnUpdate_Click;
    btnDelete.Click += BtnDelete_Click;
    btnClear.Click += BtnClear_Click;
    dgvProducts.CellClick += DgvProducts_CellClick;
    txtSearch.TextChanged += TxtSearch_TextChanged;
}

private void TxtSearch_TextChanged(object? sender, EventArgs e)
```

```

{
    string search = txtSearch.Text.Trim().ToLower();

    dgvProducts.DataSource = null;
    dgvProducts.DataSource = service.GetAll()
        .Where(p => p.Code.ToLower().Contains(search) || p.Name.ToLower().Contains(search))
        .ToList();
}

```

Este ejercicio permite relacionar listas, expresiones lambda, Contains y eventos visuales. Cada vez que el usuario escribe una letra, la tabla se actualiza con los productos que coinciden.

16. Laboratorio guiado 3: agregar categorias con ComboBox

Un ComboBox permite seleccionar una opción de una lista. Es útil para categorías, estados, tipos de documento, géneros, departamentos o cualquier dato con opciones predefinidas.

```

ComboBox cmbCategory = new ComboBox();
cmbCategory.Items.Add("Tecnologia");
cmbCategory.Items.Add("Papeleria");
cmbCategory.Items.Add("Limpieza");
cmbCategory.DropDownStyle = ComboBoxStyle.DropDownList;
cmbCategory.SelectedIndex = 0;

```

Para integrar categorías al proyecto, se debe agregar una propiedad Category al modelo Product, un campo visual ComboBox en el formulario y modificar el guardado en archivo para incluir el nuevo dato.

17. Errores comunes y solución

Error	Causa probable	Solución
No aparece el formulario	Program.cs abre otro formulario	Verificar Application.Run(new MainForm()).
No reconoce Form o Button	Falta referencia a Windows Forms	Confirmar que el proyecto sea winforms.
La tabla no se actualiza	No se llama RefreshTable	Llamar RefreshTable después de agregar, actualizar o eliminar.
Se cierra al convertir precio	Se uso decimal.Parse sin validar	Usar decimal.TryParse.
No se guardan datos	No existe carpeta Data o no se llama SaveToFile	Crear Directory.CreateDirectory("Data")
No se selecciona fila	No se configuro SelectionMode	Usar FullRowSelect en DataGridView.

18. Ejercicios propuestos

Ejercicio 1: Sistema de estudiantes

Crear una aplicación gráfica que registre estudiantes con carnet, nombre, edad y promedio. Debe mostrar los datos en una tabla, validar que el promedio esté entre 0 y 10, permitir eliminar y actualizar estudiantes.

Ejercicio 2: Control de empleados

Crear una interfaz para registrar empleados con código, nombre, cargo y salario. Debe calcular el salario anual y mostrarlo como propiedad calculada.

Ejercicio 3: Inventario con categorías

Mejorar el sistema de productos agregando categoría, proveedor, fecha de ingreso y búsqueda por categoría.

Ejercicio 4: Login básico

Crear una ventana de inicio de sesión con usuario y contraseña. Si los datos son correctos, abrir el formulario principal. Si son incorrectos, mostrar mensaje de error.

Ejercicio 5: Proyecto integrador

Crear un sistema de ventas sencillo con formularios para productos, clientes y ventas. Debe aplicar clases, listas, archivos, formularios, validaciones y DataGridView.

19. Rubrica de evaluación

Criterio	Excelente	Aceptable	Debe mejorar
Interfaz visual	Ordenada, clara y fácil de usar	Funcional pero con detalles visuales	Confusa o incompleta
Eventos	Todos los botones funcionan correctamente	La mayoría funciona	Eventos incompletos o erróneos
Validaciones	Controla campos vacíos, números y duplicados	Valida algunos casos	Permite datos incorrectos
POO	Usa modelos y servicios correctamente	Usa clases pero mezcla responsabilidades	Todo está en el formulario
Persistencia	Guarda y carga datos correctamente	Guarda parcialmente	No conserva datos
Código	Limpio, comentado y organizado	Entendible con algunos errores	Desordenado o difícil de leer

20. Recomendaciones finales

El aprendizaje de interfaces gráficas debe hacerse de forma progresiva. Primero se debe comprender cómo colocar controles y responder a eventos. Luego se agregan validaciones. Después se separa el código en clases y servicios. Finalmente se conecta con archivos o bases de datos. Este orden evita que el estudiante se pierda intentando hacer una aplicación grande sin comprender la base.

Windows Forms es una herramienta muy útil para unir la programación orientada a objetos con sistemas reales. Aunque su apariencia puede ser sencilla, permite practicar fundamentos profesionales: eventos, formularios, validaciones, tablas, separación de responsabilidades, persistencia y mantenimiento del código.

Una buena interfaz no es la que tiene más colores o más botones, sino la que permite al usuario completar una tarea sin confusión y sin cometer errores fácilmente.

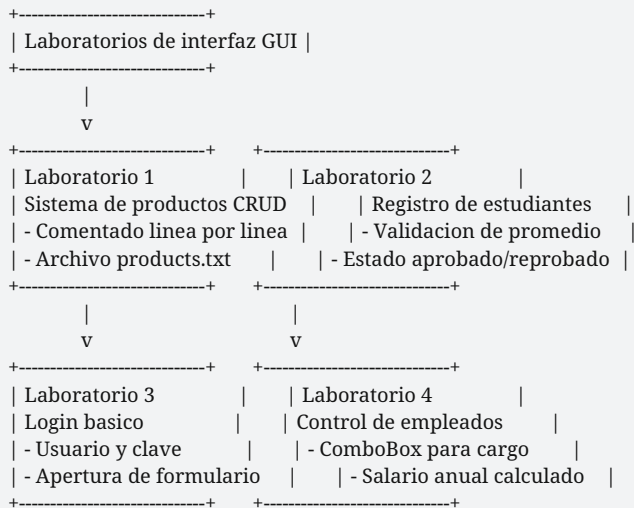
21. Actualizacion solicitada: laboratorios con codigo funcional

En esta version se amplian los laboratorios para que el estudiante no solo reciba la indicacion del ejercicio, sino tambien un codigo funcional que pueda copiar, pegar, ejecutar y modificar. El primer laboratorio, correspondiente al sistema de productos, se presenta comentado linea por linea para explicar la funcion de cada bloque, control, evento y metodo.

Importante para el instructor

Los codigos se entregan con una estructura pensada para estudiantes que vienen de consola, clases, listas, archivos y programacion orientada a objetos. El objetivo no es ocultar la logica en el diseñador visual, sino mostrar claramente como una ventana se conecta con modelos, servicios y eventos.

21.1 Diagrama general de los laboratorios



22. Laboratorio 1: Sistema de productos CRUD comentado linea por linea

Este laboratorio crea una aplicacion Windows Forms funcional para registrar productos. Incluye codigo, nombre, precio, existencia, valor total, busqueda, agregado, actualizacion, eliminacion, limpieza de campos y guardado en archivo de texto. La solucion se divide en Program.cs, Product.cs, ProductService.cs y MainForm.cs.

Archivo	Responsabilidad
Program.cs	Inicia la aplicacion grafica y abre MainForm.
Product.cs	Define los datos del producto.
ProductService.cs	Administra la lista, validaciones de negocio y archivo products.txt.
MainForm.cs	Construye la ventana, captura datos y ejecuta eventos de botones.

22.1 Comandos para crear el proyecto

```

dotnet new winforms -n SistemaProductosGUI
cd SistemaProductosGUI
mkdir Models Services Forms
  
```

22.2 Program.cs comentado

```
// Importa el namespace donde esta el formulario principal.
```

```

using SistemaProductosGUI.Forms;

// Define el namespace general del proyecto.
namespace SistemaProductosGUI;

// Clase principal del programa. Es internal porque solo se usa dentro del proyecto.
internal static class Program
{
    // [STAThread] es necesario en aplicaciones Windows Forms porque varios controles
    // visuales de Windows trabajan con un modelo de hilo de apartamento unico.
    [STAThread]
    static void Main()
    {
        // Inicializa la configuracion moderna de Windows Forms en .NET.
        ApplicationConfiguration.Initialize();

        // Abre el formulario principal y mantiene la aplicacion viva mientras la ventana este abierta.
        Application.Run(new MainForm());
    }
}

```

22.3 Models/Product.cs comentado

```

// Indica que esta clase pertenece a la carpeta logica Models.
namespace SistemaProductosGUI.Models;

// La clase Product representa un producto del inventario.
// No contiene botones ni controles visuales; solo datos.
public class Product
{
    //Codigo unico del producto. Ejemplo: P001, A100, TEC-01.
    public string Code { get; set; } = string.Empty;

    // Nombre descriptivo del producto. Ejemplo: Mouse USB.
    public string Name { get; set; } = string.Empty;

    // Precio unitario del producto. Se usa decimal porque representa dinero.
    public decimal Price { get; set; }

    // Cantidad disponible en inventario.
    public int Stock { get; set; }

    // Propiedad calculada: no se captura directamente, se obtiene multiplicando precio por existencia.
    public decimal TotalValue => Price * Stock;
}

```

22.4 Services/ProductService.cs comentado

```

// Permite usar la clase Product.
using SistemaProductosGUI.Models;

// Permite trabajar con CultureInfo para guardar decimales con punto y no depender del idioma del
// sistema.
using System.Globalization;

// Indica que esta clase pertenece a la carpeta logica Services.
namespace SistemaProductosGUI.Services;

// ProductService se encarga de administrar la lista de productos.
// La interfaz grafica llamara a este servicio para agregar, actualizar, eliminar y guardar datos.
public class ProductService
{
    // Lista privada donde se almacenan los productos mientras el programa esta abierto.
    private readonly List<Product> products = new();
}

```

```
// Ruta del archivo donde se guardaran los productos.
private readonly string filePath = Path.Combine(AppDomain.CurrentDomain.BaseDirectory,
"products.txt");

// Constructor del servicio. Cuando se crea el servicio, intenta cargar datos del archivo.
public ProductService()
{
    LoadFromFile();
}

// Devuelve una copia de la lista para que el formulario pueda mostrar los productos.
public List<Product> GetAll()
{
    return products.ToList();
}

// Busca productos por codigo o nombre. Se usa para el laboratorio de busqueda.
public List<Product> Search(string text)
{
    // Si no se escribe nada, se devuelven todos los productos.
    if (string.IsNullOrEmpty(text))
        return GetAll();

    // Convierte el texto a minusculas para hacer una busqueda sin distinguir mayusculas.
    string filter = text.Trim().ToLower();

    // Where filtra la lista. Contains revisa si el codigo o nombre contiene el texto buscado.
    return products
        .Where(p => p.Code.ToLower().Contains(filter) || p.Name.ToLower().Contains(filter))
        .ToList();
}

// Agrega un nuevo producto a la lista.
public void Add(Product product)
{
    // Any revisa si ya existe un producto con el mismo codigo.
    bool exists = products.Any(p => p.Code.Equals(product.Code, StringComparison.OrdinalIgnoreCase));

    // Si ya existe, se lanza una excepcion para avisar al formulario.
    if (exists)
        throw new InvalidOperationException("Ya existe un producto con ese codigo.");

    // Si no existe, se agrega a la lista.
    products.Add(product);

    // Guarda los cambios en el archivo.
    SaveToFile();
}

// Actualiza un producto existente.
public void Update(Product product)
{
    // Busca el producto original por codigo.
    Product? existing = products.FirstOrDefault(p => p.Code.Equals(product.Code,
StringComparison.OrdinalIgnoreCase));

    // Si no se encuentra, no se puede actualizar.
    if (existing == null)
        throw new InvalidOperationException("No se encontro el producto a actualizar.");

    // Actualiza los datos editables.
    existing.Name = product.Name;
    existing.Price = product.Price;
    existing.Stock = product.Stock;

    // Guarda los cambios en archivo.
    SaveToFile();
}
```



```

}

// Elimina un producto por codigo.
public void Delete(string code)
{
    // Busca el producto que se desea eliminar.
    Product? existing = products.FirstOrDefault(p => p.Code.Equals(code,
StringComparison.OrdinalIgnoreCase));

    // Si no existe, se lanza una excepcion.
    if (existing == null)
        throw new InvalidOperationException("No se encontro el producto a eliminar.");

    // Remueve el producto de la lista.
    products.Remove(existing);

    // Guarda los cambios en archivo.
    SaveToFile();
}

// Guarda todos los productos en un archivo de texto separado por punto y coma.
private void SaveToFile()
{
    // Select transforma cada producto en una linea de texto.
    IEnumerable<string> lines = products.Select(p =>
        $"{p.Code};{p.Name};{p.Price.ToString(CultureInfo.InvariantCulture)};{p.Stock}");

    // WriteAllLines crea o reemplaza el archivo con las lineas generadas.
    File.WriteAllLines(filePath, lines);
}

// Carga productos desde el archivo si existe.
private void LoadFromFile()
{
    // Si el archivo aun no existe, no hay nada que cargar.
    if (!File.Exists(filePath))
        return;

    // Recorre cada linea del archivo.
    foreach (string line in File.ReadAllLines(filePath))
    {
        // Divide la linea usando punto y coma.
        string[] parts = line.Split(';');

        // Si la linea no tiene las 4 partes esperadas, se ignora.
        if (parts.Length != 4)
            continue;

        // Convierte el precio usando cultura invariante.
        bool validPrice = decimal.TryParse(parts[2], NumberStyles.Any, CultureInfo.InvariantCulture,
out decimal price);

        // Convierte la existencia a entero.
        bool validStock = int.TryParse(parts[3], out int stock);

        // Si las conversiones fallan, se ignora la linea.
        if (!validPrice || !validStock)
            continue;

        // Agrega el producto cargado a la lista.
        products.Add(new Product
        {
            Code = parts[0],
            Name = parts[1],
            Price = price,
            Stock = stock
        });
    }
}

```

```

    }
}
}

```

22.5 Forms/MainForm.cs comentado

```

// Permite usar el modelo Product.
using SistemaProductosGUI.Models;

// Permite usar el servicio ProductService.
using SistemaProductosGUI.Services;

// Indica que esta clase pertenece a la carpeta logica Forms.
namespace SistemaProductosGUI.Forms;

// MainForm hereda de Form, por eso representa una ventana de Windows Forms.
public class MainForm : Form
{
    // Servicio que maneja la logica de productos.
    private readonly ProductService service = new();

    // Etiquetas que describen los campos.
    private readonly Label lblCode = new() { Text = "Codigo:", Left = 20, Top = 25, Width = 100 };
    private readonly Label lblName = new() { Text = "Nombre:", Left = 20, Top = 65, Width = 100 };
    private readonly Label lblPrice = new() { Text = "Precio:", Left = 20, Top = 105, Width = 100 };
    private readonly Label lblStock = new() { Text = "Existencia:", Left = 20, Top = 145, Width = 100 };
    private readonly Label lblSearch = new() { Text = "Buscar:", Left = 20, Top = 185, Width = 100 };

    // Cajas de texto para capturar datos.
    private readonly TextBox txtCode = new() { Left = 130, Top = 20, Width = 220 };
    private readonly TextBox txtName = new() { Left = 130, Top = 60, Width = 220 };
    private readonly TextBox txtPrice = new() { Left = 130, Top = 100, Width = 220 };
    private readonly TextBox txtStock = new() { Left = 130, Top = 140, Width = 220 };
    private readonly TextBox txtSearch = new() { Left = 130, Top = 180, Width = 220 };

    // Botones de acciones CRUD.
    private readonly Button btnAdd = new() { Text = "Agregar", Left = 380, Top = 20, Width = 120 };
    private readonly Button btnUpdate = new() { Text = "Actualizar", Left = 380, Top = 60, Width = 120 };
    private readonly Button btnDelete = new() { Text = "Eliminar", Left = 380, Top = 100, Width = 120 };
    private readonly Button btnClear = new() { Text = "Limpiar", Left = 380, Top = 140, Width = 120 };

    // Tabla visual donde se muestran los productos.
    private readonly DataGridView dgvProducts = new()
    {
        Left = 20,
        Top = 230,
        Width = 760,
        Height = 300,
        ReadOnly = true,
        AutoSizeColumnsMode = DataGridViewAutoSizeColumnsMode.Fill,
        SelectionMode = DataGridViewSelectionMode.FullRowSelect,
        MultiSelect = false
    };

    // Constructor del formulario. Se ejecuta al crear la ventana.
    public MainForm()
    {
        // Configura propiedades generales de la ventana.
        Text = "Sistema de productos";
        Width = 830;
        Height = 600;
        StartPosition = FormStartPosition.CenterScreen;
        FormBorderStyle = FormBorderStyle.FixedSingle;
        MaximizeBox = false;

        // Agrega todos los controles visuales al formulario.

```

```

Controls.AddRange(new Control[]
{
    lblCode, lblName, lblPrice, lblStock, lblSearch,
    txtCode, txtName, txtPrice, txtStock, txtSearch,
    btnAdd, btnUpdate, btnDelete, btnClear,
    dgvProducts
});

// Conecta cada boton con su evento correspondiente.
btnAdd.Click += BtnAdd_Click;
btnUpdate.Click += BtnUpdate_Click;
btnDelete.Click += BtnDelete_Click;
btnClear.Click += BtnClear_Click;

// Conecta la tabla con el evento que llena los campos al seleccionar una fila.
dgvProducts.CellClick += DgvProducts_CellClick;

// Cada vez que se escribe en buscar, se filtra la tabla.
txtSearch.TextChanged += TxtSearch_TextChanged;

// Carga los productos iniciales en la tabla.
RefreshGrid();
}

// Evento del boton Agregar.
private void BtnAdd_Click(object? sender, EventArgs e)
{
    try
    {
        // Lee y valida los datos escritos por el usuario.
        Product product = ReadProductFromForm();

        // Agrega el producto usando el servicio.
        service.Add(product);

        // Actualiza la tabla.
        RefreshGrid();

        // Limpia los campos.
        ClearForm();

        // Muestra mensaje de exito.
        MessageBox.Show("Producto agregado correctamente.");
    }
    catch (Exception ex)
    {
        // Muestra cualquier error de validacion o regla de negocio.
        MessageBox.Show(ex.Message, "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}

// Evento del boton Actualizar.
private void BtnUpdate_Click(object? sender, EventArgs e)
{
    try
    {
        Product product = ReadProductFromForm();
        service.Update(product);
        RefreshGrid();
        ClearForm();
        MessageBox.Show("Producto actualizado correctamente.");
    }
    catch (Exception ex)
    {
        MessageBox.Show(ex.Message, "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}

```

```
// Evento del boton Eliminar.
private void BtnDelete_Click(object? sender, EventArgs e)
{
    try
    {
        // Valida que el codigo no este vacio.
        if (string.IsNullOrEmpty(txtCode.Text))
            throw new Exception("Seleccione o escriba el codigo del producto a eliminar.");

        // Pide confirmacion antes de eliminar.
        DialogResult result = MessageBox.Show(
            "¿Esta seguro de eliminar este producto?",
            "Confirmar eliminacion",
            MessageBoxButtons.YesNo,
            MessageBoxIcon.Question);

        // Si el usuario responde No, se cancela la accion.
        if (result == DialogResult.No)
            return;

        // Elimina el producto usando el servicio.
        service.Delete(txtCode.Text.Trim());

        // Refresca la tabla y limpia los campos.
        RefreshGrid();
        ClearForm();

        MessageBox.Show("Producto eliminado correctamente.");
    }
    catch (Exception ex)
    {
        MessageBox.Show(ex.Message, "Error", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}

// Evento del boton Limpiar.
private void BtnClear_Click(object? sender, EventArgs e)
{
    ClearForm();
}

// Evento que se ejecuta cuando se hace clic en una fila de la tabla.
private void DgvProducts_CellClick(object? sender, DataGridViewCellEventArgs e)
{
    // Evita errores si se hace clic en el encabezado.
    if (e.RowIndex < 0)
        return;

    // Obtiene el producto seleccionado.
    Product? selected = dgvProducts.Rows[e.RowIndex].DataBoundItem as Product;

    // Si no se pudo convertir, se cancela.
    if (selected == null)
        return;

    // Copia los datos de la tabla a los campos.
    txtCode.Text = selected.Code;
    txtName.Text = selected.Name;
    txtPrice.Text = selected.Price.ToString();
    txtStock.Text = selected.Stock.ToString();
}

// Evento que filtra la tabla mientras el usuario escribe en el buscador.
private void TxtSearch_TextChanged(object? sender, EventArgs e)
{
    dgvProducts.DataSource = null;
```

```

dgvProducts.DataSource = service.Search(txtSearch.Text);
}

// Lee los controles, valida datos y devuelve un objeto Product.
private Product ReadProductFromForm()
{
    // Valida código obligatorio.
    if (string.IsNullOrEmpty(txtCode.Text))
        throw new Exception("El código es obligatorio.");

    // Valida nombre obligatorio.
    if (string.IsNullOrEmpty(txtName.Text))
        throw new Exception("El nombre es obligatorio.");

    // Convierte el precio a decimal.
    if (!decimal.TryParse(txtPrice.Text, out decimal price) || price <= 0)
        throw new Exception("El precio debe ser un número mayor que cero.");

    // Convierte la existencia a entero.
    if (!int.TryParse(txtStock.Text, out int stock) || stock < 0)
        throw new Exception("La existencia debe ser un número entero mayor o igual que cero.");

    // Devuelve el producto construido con los datos capturados.
    return new Product
    {
        Code = txtCode.Text.Trim(),
        Name = txtName.Text.Trim(),
        Price = price,
        Stock = stock
    };
}

// Recarga la tabla con todos los productos.
private void RefreshGrid()
{
    dgvProducts.DataSource = null;
    dgvProducts.DataSource = service.GetAll();
}

// Limpia los campos del formulario.
private void ClearForm()
{
    txtCode.Clear();
    txtName.Clear();
    txtPrice.Clear();
    txtStock.Clear();
    txtSearch.Clear();
    txtCode.Focus();
}
}

```

22.6 Resultado esperado del laboratorio 1

Al ejecutar la aplicación debe abrirse una ventana con campos para código, nombre, precio, existencia y búsqueda. El estudiante debe poder agregar productos, seleccionarlos desde la tabla, actualizarlos, eliminarlos y comprobar que al cerrar y volver a abrir el programa los datos se conservan en el archivo products.txt.

23. Laboratorio 2: Registro de estudiantes con promedio y estado

Este laboratorio refuerza el uso de TextBox, Button, DataGridView, validaciones numéricas y propiedades calculadas. El sistema registra estudiantes con carnet, nombre, edad y promedio. El estado se calcula automáticamente según el promedio.

23.1 Codigo funcional completo

```
// Proyecto sugerido: dotnet new winforms -n EstudiantesGUI
// Archivo: Program.cs
namespace EstudiantesGUI;

internal static class Program
{
    [STAThread]
    static void Main()
    {
        ApplicationConfiguration.Initialize();
        Application.Run(new MainForm());
    }
}

// Archivo: Student.cs
namespace EstudiantesGUI;

public class Student
{
    public string Card { get; set; } = string.Empty;
    public string Name { get; set; } = string.Empty;
    public int Age { get; set; }
    public decimal Average { get; set; }
    public string Status => Average >= 6 ? "Aprobado" : "Reprobado";
}

// Archivo: StudentService.cs
namespace EstudiantesGUI;

public class StudentService
{
    private readonly List<Student> students = new();

    public List<Student> GetAll() => students.ToList();

    public void Add(Student student)
    {
        if (students.Any(s => s.Card.Equals(student.Card, StringComparison.OrdinalIgnoreCase)))
            throw new Exception("Ya existe un estudiante con ese carnet.");
        students.Add(student);
    }

    public void Update(Student student)
    {
        Student? existing = students.FirstOrDefault(s => s.Card.Equals(student.Card,
            StringComparison.OrdinalIgnoreCase));
        if (existing == null) throw new Exception("No se encontro el estudiante.");
        existing.Name = student.Name;
        existing.Age = student.Age;
        existing.Average = student.Average;
    }

    public void Delete(string card)
    {
        Student? existing = students.FirstOrDefault(s => s.Card.Equals(card,
            StringComparison.OrdinalIgnoreCase));
        if (existing == null) throw new Exception("No se encontro el estudiante.");
        students.Remove(existing);
    }
}

// Archivo: MainForm.cs
namespace EstudiantesGUI;
```

```

public class MainForm : Form
{
    private readonly StudentService service = new();
    private readonly TextBox txtCard = new() { Left = 120, Top = 20, Width = 200 };
    private readonly TextBox txtName = new() { Left = 120, Top = 60, Width = 200 };
    private readonly TextBox txtAge = new() { Left = 120, Top = 100, Width = 200 };
    private readonly TextBox txtAverage = new() { Left = 120, Top = 140, Width = 200 };
    private readonly Button btnAdd = new() { Text = "Agregar", Left = 350, Top = 20, Width = 100 };
    private readonly Button btnUpdate = new() { Text = "Actualizar", Left = 350, Top = 60, Width = 100 };
    private readonly Button btnDelete = new() { Text = "Eliminar", Left = 350, Top = 100, Width = 100 };
    private readonly Button btnClear = new() { Text = "Limpiar", Left = 350, Top = 140, Width = 100 };
    private readonly DataGridView dgv = new() { Left = 20, Top = 200, Width = 650, Height = 250, ReadOnly
    = true, AutoSizeColumnsMode = DataGridViewAutoSizeColumnsMode.Fill };

    public MainForm()
    {
        Text = "Registro de estudiantes";
        Width = 720;
        Height = 520;
        StartPosition = FormStartPosition.CenterScreen;
        Controls.AddRange(new Control[] {
            new Label { Text = "Carnet:", Left = 20, Top = 25, Width = 90 }, txtCard,
            new Label { Text = "Nombre:", Left = 20, Top = 65, Width = 90 }, txtName,
            new Label { Text = "Edad:", Left = 20, Top = 105, Width = 90 }, txtAge,
            new Label { Text = "Promedio:", Left = 20, Top = 145, Width = 90 }, txtAverage,
            btnAdd, btnUpdate, btnDelete, btnClear, dgv
        });
        btnAdd.Click += (_, _) => Execute() => { service.Add(ReadStudent()); RefreshGrid(); ClearForm();
    };
        btnUpdate.Click += (_, _) => Execute() => { service.Update(ReadStudent()); RefreshGrid();
    ClearForm(); };
        btnDelete.Click += (_, _) => Execute() => { service.Delete(txtCard.Text.Trim()); RefreshGrid();
    ClearForm(); };
        btnClear.Click += (_, _) => ClearForm();
        dgv.CellClick += Dgv_CellClick;
        RefreshGrid();
    }

    private Student ReadStudent()
    {
        if (string.IsNullOrEmpty(txtCard.Text)) throw new Exception("El carnet es obligatorio.");
        if (string.IsNullOrEmpty(txtName.Text)) throw new Exception("El nombre es obligatorio.");
        if (!int.TryParse(txtAge.Text, out int age) || age < 15 || age > 80) throw new Exception("Edad
    invalida.");
        if (!decimal.TryParse(txtAverage.Text, out decimal average) || average < 0 || average > 10) throw
    new Exception("Promedio invalido.");
        return new Student { Card = txtCard.Text.Trim(), Name = txtName.Text.Trim(), Age = age, Average =
    average };
    }

    private void Dgv_CellClick(object? sender, DataGridViewCellEventArgs e)
    {
        if (e.RowIndex < 0) return;
        if (dgv.Rows[e.RowIndex].DataBoundItem is not Student s) return;
        txtCard.Text = s.Card;
        txtName.Text = s.Name;
        txtAge.Text = s.Age.ToString();
        txtAverage.Text = s.Average.ToString();
    }

    private void Execute(Action action)
    {
        try { action(); MessageBox.Show("Operacion realizada correctamente."); }
        catch (Exception ex) { MessageBox.Show(ex.Message, "Error", MessageBoxButtons.OK,
    MessageBoxIcon.Error); }
    }

    private void RefreshGrid() { dgv.DataSource = null; dgv.DataSource = service.GetAll(); }

```

```
private void ClearForm() { txtCard.Clear(); txtName.Clear(); txtAge.Clear(); txtAverage.Clear();
txtCard.Focus(); }
}
```

23.2 Pruebas sugeridas

Probar un estudiante con promedio 8.5 y verificar que aparezca como Aprobado. Luego probar promedio 5.2 y verificar que aparezca como Reprobado. Intentar ingresar una edad no numerica y confirmar que el sistema muestra el mensaje de error.

24. Laboratorio 3: Login basico y apertura de formulario principal

Este laboratorio permite practicar multiples formularios. El primer formulario solicita usuario y contrasena. Si los datos son correctos, se oculta el formulario de login y se muestra una ventana principal. Para fines didacticos, el usuario sera admin y la contrasena sera 1234.

24.1 Codigo funcional completo

```
// Proyecto sugerido: dotnet new winforms -n LoginBasicoGUI
// Archivo: Program.cs
namespace LoginBasicoGUI;

internal static class Program
{
    [STAThread]
    static void Main()
    {
        ApplicationConfiguration.Initialize();
        Application.Run(new LoginForm());
    }
}

// Archivo: LoginForm.cs
namespace LoginBasicoGUI;

public class LoginForm : Form
{
    private readonly TextBox txtUser = new() { Left = 130, Top = 35, Width = 180 };
    private readonly TextBox txtPassword = new() { Left = 130, Top = 80, Width = 180, PasswordChar = '*' };
    private readonly Button btnLogin = new() { Text = "Ingresar", Left = 130, Top = 125, Width = 180 };

    public LoginForm()
    {
        Text = "Inicio de sesion";
        Width = 380;
        Height = 230;
        StartPosition = FormStartPosition.CenterScreen;
        Controls.AddRange(new Control[] {
            new Label { Text = "Usuario:", Left = 30, Top = 40, Width = 90 }, txtUser,
            new Label { Text = "Clave:", Left = 30, Top = 85, Width = 90 }, txtPassword,
            btnLogin
        });
        btnLogin.Click += BtnLogin_Click;
    }

    private void BtnLogin_Click(object? sender, EventArgs e)
    {
        string user = txtUser.Text.Trim();
        string password = txtPassword.Text.Trim();
        if (user == "admin" && password == "1234")
        {
            Hide();
        }
    }
}
```



```

        MainForm main = new MainForm();
        main.FormClosed += (_, _) => Close();
        main.Show();
    }
    else
    {
        MessageBox.Show("Usuario o contrasena incorrectos.", "Acceso denegado", MessageBoxButtons.OK,
        MessageBoxIcon.Warning);
        txtPassword.Clear();
        txtPassword.Focus();
    }
}
}

// Archivo: MainForm.cs
namespace LoginBasicoGUI;

public class MainForm : Form
{
    public MainForm()
    {
        Text = "Panel principal";
        Width = 500;
        Height = 300;
        StartPosition = FormStartPosition.CenterScreen;
        Controls.Add(new Label
        {
            Text = "Bienvenido al sistema",
            Left = 140,
            Top = 100,
            Width = 250,
            Font = new Font("Arial", 16, FontStyle.Bold)
        });
    }
}

```

24.2 Pruebas sugeridas

Ingresar credenciales incorrectas y verificar que el sistema limpie la contrasena. Luego ingresar admin y 1234 para comprobar que se abre el panel principal. Explicar que en sistemas reales la contrasena no debe escribirse fija en el codigo, sino verificarse contra una base de datos o servicio seguro.

25. Laboratorio 4: Control de empleados con ComboBox y propiedad calculada

Este laboratorio agrega un ComboBox para seleccionar cargo y una propiedad calculada para el salario anual. Permite relacionar controles visuales con clases y listas, manteniendo una logica sencilla para estudiantes que estan iniciando en interfaces graficas.

25.1 Codigo funcional completo

```

// Proyecto sugerido: dotnet new winforms -n EmpleadosGUI
// Este laboratorio usa ComboBox para cargos y calcula salario anual.
namespace EmpleadosGUI;

internal static class Program
{
    [STAThread]
    static void Main()
    {
        ApplicationConfiguration.Initialize();
        Application.Run(new MainForm());
    }
}

```

```

    }
}

public class Employee
{
    public string Code { get; set; } = string.Empty;
    public string Name { get; set; } = string.Empty;
    public string Position { get; set; } = string.Empty;
    public decimal MonthlySalary { get; set; }
    public decimal AnnualSalary => MonthlySalary * 12;
}

public class MainForm : Form
{
    private readonly List<Employee> employees = new();
    private readonly TextBox txtCode = new() { Left = 130, Top = 20, Width = 200 };
    private readonly TextBox txtName = new() { Left = 130, Top = 60, Width = 200 };
    private readonly ComboBox cmbPosition = new() { Left = 130, Top = 100, Width = 200, DropDownStyle =
        ComboBoxStyle.DropDownList };
    private readonly TextBox txtSalary = new() { Left = 130, Top = 140, Width = 200 };
    private readonly DataGridView dgv = new() { Left = 20, Top = 220, Width = 680, Height = 250, ReadOnly
        = true, AutoSizeColumnsMode = DataGridViewAutoSizeColumnsMode.Fill };

    public MainForm()
    {
        Text = "Control de empleados";
        Width = 760;
        Height = 540;
        StartPosition = FormStartPosition.CenterScreen;
        cmbPosition.Items.AddRange(new object[] { "Programador", "Soporte", "Analista", "Administrador"
    });
        Button btnAdd = new() { Text = "Agregar", Left = 370, Top = 20, Width = 110 };
        Button btnDelete = new() { Text = "Eliminar", Left = 370, Top = 60, Width = 110 };
        Button btnClear = new() { Text = "Limpiar", Left = 370, Top = 100, Width = 110 };
        Controls.AddRange(new Control[] {
            new Label { Text = "Codigo:", Left = 20, Top = 25, Width = 100 }, txtCode,
            new Label { Text = "Nombre:", Left = 20, Top = 65, Width = 100 }, txtName,
            new Label { Text = "Cargo:", Left = 20, Top = 105, Width = 100 }, cmbPosition,
            new Label { Text = "Salario mensual:", Left = 20, Top = 145, Width = 110 }, txtSalary,
            btnAdd, btnDelete, btnClear, dgv
        });
        btnAdd.Click += (_, _) => AddEmployee();
        btnDelete.Click += (_, _) => DeleteEmployee();
        btnClear.Click += (_, _) => ClearForm();
        dgv.CellClick += Dgv_CellClick;
    }

    private void AddEmployee()
    {
        if (string.IsNullOrEmpty(txtCode.Text) || string.IsNullOrEmpty(txtName.Text)) {
            MessageBox.Show("Complete codigo y nombre."); return; }
        if (cmbPosition.SelectedIndex == -1) { MessageBox.Show("Seleccione un cargo."); return; }
        if (!decimal.TryParse(txtSalary.Text, out decimal salary) || salary <= 0) {
            MessageBox.Show("Salario invalido."); return; }
        if (employees.Any(e => e.Code == txtCode.Text.Trim())) { MessageBox.Show("Codigo repetido.");
            return; }
        employees.Add(new Employee { Code = txtCode.Text.Trim(), Name = txtName.Text.Trim(), Position =
            cmbPosition.Text, MonthlySalary = salary });
        RefreshGrid();
        ClearForm();
    }

    private void DeleteEmployee()
    {
        Employee? employee = employees.FirstOrDefault(e => e.Code == txtCode.Text.Trim());
        if (employee == null) { MessageBox.Show("Seleccione un empleado."); return; }
        employees.Remove(employee);
        RefreshGrid();
    }
}

```

```

ClearForm();
}

private void Dgv_CellClick(object? sender, DataGridViewCellEventArgs e)
{
    if (e.RowIndex < 0) return;
    if (dgv.Rows[e.RowIndex].DataBoundItem is not Employee emp) return;
    txtCode.Text = emp.Code;
    txtName.Text = emp.Name;
    cmbPosition.Text = emp.Position;
    txtSalary.Text = emp.MonthlySalary.ToString();
}

private void RefreshGrid() { dgv.DataSource = null; dgv.DataSource = employees.ToList(); }
private void ClearForm() { txtCode.Clear(); txtName.Clear(); txtSalary.Clear();
    cmbPosition.SelectedIndex = -1; txtCode.Focus(); }
}

```

25.2 Pruebas sugeridas

Registrar un empleado con salario mensual de 650 y verificar que el salario anual sea 7800. Probar dejar el cargo vacio para comprobar la validacion del ComboBox. Seleccionar una fila de la tabla y eliminar el empleado.

26. Forma recomendada de evaluar los laboratorios

Criterio	Descripcion	Puntaje sugerido
Ejecucion	El proyecto compila y abre la ventana sin errores.	20%
Interfaz	Los controles estan ordenados y tienen nombres comprensibles.	15%
Validaciones	El sistema evita campos vacios y datos numericos incorrectos.	20%
Eventos	Los botones ejecutan correctamente agregar, actualizar, eliminar y limpiar.	20%
POO	Se usan clases, listas, propiedades y separacion de responsabilidades.	15%
Explicacion	El estudiante puede explicar que hace cada archivo y cada evento principal.	10%

Cierre didactico

Estos laboratorios deben trabajarse copiando primero el codigo funcional y luego modificandolo. La modificacion es clave: cambiar nombres de campos, agregar una propiedad, crear una nueva validacion o adaptar el proyecto a otro caso permite comprobar que el estudiante realmente comprendio la estructura.